



UNIVERSITÀ
DEGLI STUDI
FIRENZE

SOFTWARE PERFORMANCE ENGINEERING, 2026

**Performance Modelling of a Microservice Application:
Analytical Queueing Network Formulation and Validation**
Target Benchmark: Google Online Boutique (11 Microservices)

By

Essah Janneh & Opoka John Modi Silvio

May 25th, 2025

Abstract

Modern cloud-native applications are constructed as complex directed acyclic graphs of loosely coupled microservices. Predicting their performance under load is a critical challenge in software capacity planning, automated scaling, and predictive bottleneck mitigation.

This report presents a complete, validated Open Queueing Network (QN) model of the Google Online Boutique microservice benchmark (11 services) solved via Mean Value Analysis (MVA), extended programmatically to M/M/c multi-server scaling. The model parameters — comprising mean service demands and transition routing probabilities — were empirically characterized under steady-state conditions using a custom observability stack incorporating OpenTelemetry (OTel), Prometheus, cAdvisor, and Jaeger distributed tracing.

The analytical model demonstrated high accuracy in identifying the primary system bottleneck (Frontend service, 3.1% relative prediction error) and provided system-level response time predictions within an acceptable range. Key accuracy metrics: RMSE = 4.98 ms, MAE = 4.52 ms, MAPE = 33.34% across all service centers. Horizontal scaling strategies were mathematically modeled and verified via a REST API, confirming that replicating the Frontend to $c_0 = 2$ containers reduces its queue delay from an unbounded, saturated state to a stable response time of 174.8 ms at 10 req/s.

Table of Content

Abstract	ii
Table of Content	iii
1.0 Introduction & Objectives.....	1
1.1 Core Project Objectives.....	1
2.0 Target System Architecture	2
2.1 Microservice Directory	2
2.2 Deployment Environment	3
3.0 Queueing Network Model Formulation.....	3
3.1 Mathematical Definitions	3
3.2 Traffic Equations and Visit Ratios	3
3.3 Per-Service Performance Metrics (M/M/1).....	4
3.4 System-Level Performance Metrics	5
4.0 Multi-Server Scaling: M/M/c Queueing	5
4.1 M/M/c Derivations	5
4.2 Horizontal Scaling Scenario: Frontend Saturation at 10 req/s	6
5.0 Workload Characterization & Observability Stack	6
5.1 Observability Stack Architecture	6
5.2 Parameter Extraction	7
5.3 Session Transition Matrix (P)	7
6.0 Software Architecture & Implementation.....	8
6.1 Project Layout	8
6.2 Component Responsibilities.....	9
7.0 Empirical Validation & Results	9
7.1 Per-Service Response Time Validation.....	9
7.2 Compiled Error Metrics	10
7.3 Sensitivity & Saturation Sweep	11
8.0 Technical Challenges & Solutions.....	12
Challenge 1: Docker Container Namespace Resolution under WSL 2.....	12
Challenge 2: OTel Context Propagation Disconnection	12

Challenge 3: Parameter Scoping for REST API Decoupling.....	12
9.0 Replication & Reproducibility Guide	13
Stage A: End-to-End Modeling CLI	13
Stage B: REST API Scaling Predictions	13
10. Future Work & Recommendations	14
10.1 Layered Queueing Networks (LQN).....	14
10.2 Connection Pool and Thread Contention Modeling.....	14
10.3 Network Transit and Serialization Demands	14
11. Conclusions.....	15
References.....	a

1.0 Introduction & Objectives

Performance degradation in microservices is fundamentally non-linear. It is typically driven by queueing delays at shared resources — CPU cores, thread pools, database connection pools — rather than by the linear growth of raw service execution time. When a resource reaches saturation, queue lengths grow without bound and response times diverge to infinity, a behavior described by the M/M/1 stability condition $\rho < 1$.

While load testing is a common industry practice for performance validation, it is resource-intensive, slow to iterate, and cannot cheaply explore arbitrary workload scenarios or architectural topologies. Analytical queueing network (QN) models offer a complementary, mathematically rigorous alternative that can be evaluated in milliseconds — enabling rapid "what-if" capacity planning, bottleneck identification, and scaling decisions.

1.1 Core Project Objectives

1. System Deployment: Deploy the 11-service Google Online Boutique application in a controlled containerized environment alongside a full monitoring stack.
2. Empirical Parameterization: Collect real-world telemetry via Prometheus and Jaeger to compute exclusive service demands (S_i) and transition routing probabilities (P_{ij}).
3. Queueing Network Formulation: Model each microservice as an independent First-Come-First-Served (FCFS) M/M/1 service center within an Open Queueing Network.
4. MVA Solver Implementation: Implement an Open-Network Mean Value Analysis solver in Python to compute throughput, utilization, queue length, and response times.
5. Validation & Sensitivity Analysis: Compare analytical predictions against empirical measurements, locate the bottleneck, and sweep arrival rates to identify the saturation point.
6. Horizontal Scale Modeling: Design and expose a REST API capable of predicting performance under horizontal pod replication using M/M/c (Erlang-C) queueing.

Key Insight

Analytical models solve in milliseconds vs. hours of load testing. A complete sweep of 1–10 req/s across 11 service centers takes under 50 ms of Python CPU time, compared to the hours required for equivalent empirical coverage using a load generator.

2.0 Target System Architecture

The target benchmark is the Google Online Boutique, an open-source, cloud-native e-commerce demo application maintained by Google Cloud Platform. It comprises 11 microservices implemented in five different programming languages (Go, C#, Node.js, Python, Java), communicating primarily over gRPC. The sole exception is the Frontend, which exposes an HTTP interface to end-users and internally translates HTTP requests into gRPC calls to downstream services.

This polyglot design intentionally mirrors real-world microservice ecosystems and introduces practical observability challenges — particularly around distributed trace context propagation across language boundaries, which required manual OTel configuration (see Section 8).

2.1 Microservice Directory

Microservice	Language	Responsibility
Frontend	Go	Serves the HTTP web UI and delegates back-end tasks to internal services.
CartService	C#	Manages user shopping carts; backed by Redis for in-memory persistence.
ProductCatalogService	Go	Retrieves product listings, descriptions, and details.
CurrencyService	Node.js	Performs price conversions across multiple currencies.
RecommendationService	Python	Recommends related products based on current cart state.
AdService	Java	Returns contextually targeted advertisements.
CheckoutService	Go	Orchestrates the multi-step checkout workflow (calls 6 downstream services).
ShippingService	Go	Calculates shipping rates for the current order.
PaymentService	Node.js	Processes mock credit card credentials.
EmailService	Python	Dispatches mock order-confirmation emails.
Redis	—	In-memory key-value store managing cart state persistence.

The call graph is hierarchical: all external traffic enters through the Frontend, which fans out to ProductCatalog, Currency, Cart, Recommendation, Ad, and Checkout. The CheckoutService in turn invokes ProductCatalog, Currency, Cart, Shipping, Payment, and Email — making it the most

downstream-coupled service in the topology. CartService holds state in Redis, the only non-HTTP/gRPC dependency.

2.2 Deployment Environment

The application was deployed using Docker Compose on a Windows host with a WSL 2 Linux engine. This configuration introduced specific namespace-resolution challenges for cAdvisor monitoring (detailed in Section 8). Each microservice ran as an independent container, with resource limits set to reflect a realistic but constrained single-node deployment for controlled profiling.

3.0 Queueing Network Model Formulation

The microservice topology is modeled as an Open Queueing Network (OQN) with $K = 11$ service centers. "Open" means external requests arrive from outside the system according to a Poisson process at rate λ and depart after receiving service — the population is not fixed, unlike closed networks. Each service center operates as an independent FCFS (First-Come-First-Served) single-server queue with exponentially distributed service times, i.e., an M/M/1 queue.

The exponential service-time assumption (M) is a classical simplification. It is justified here because the Poisson/exponential model is analytically tractable, yields closed-form solutions, and has been shown empirically to predict bottleneck behavior accurately even when the true service-time distribution is not exponential.

3.1 Mathematical Definitions

Let the following notation be established:

- $\lambda_o = [\lambda_{o,0}, \lambda_{o,1}, \dots, \lambda_{o,K-1}]^T$ — vector of external arrival rates. Since users only hit the Frontend directly: $\lambda_{o,0} = \lambda$ (total external rate) and $\lambda_{o,i} = 0$ for $i > 0$.
- P — $K \times K$ routing matrix where P_{ij} = probability that a request transitions from service i to service j upon completion.
- S_i — mean service demand (exclusive processing time) at service center i , measured in seconds.

3.2 Traffic Equations and Visit Ratios

The internal arrival rate λ_i at each service center balances external arrivals and internal routings. This is captured by the global balance (traffic) equations:

$$\lambda_i = \lambda_{o,i} + \sum_j \lambda_j \cdot P_{j,i} \quad \text{for all } i = 0, 1, \dots, K-1$$

In compact matrix form: $\lambda = \lambda_o + P^T \lambda$, which rearranges to $(I - P^T)\lambda = \lambda_o$. Solving the linear system:

$$\lambda = (I - P^T)^{-1} \lambda_o$$

The visit ratio V_i captures how many times, on average, service i is visited per single external request entering the system:

$$V_i = \lambda_i / \lambda$$

Visit ratios greater than 1 indicate that a service is called multiple times per end-to-end request (e.g., ProductCatalogService is called by both Frontend and Checkout). Visit ratios are fundamental to understanding why a lightly-loaded service can still become a bottleneck under high arrival rates.

3.3 Per-Service Performance Metrics (M/M/1)

For each service center i , three key metrics are derived from the arrival rate λ_i and service demand S_i :

Utilization (ρ_i)

The fraction of time the server is busy. This is the most critical metric for bottleneck identification:

$$\rho_i = \lambda_i \cdot S_i$$

Stability condition: $\rho_i < 1$ for all i . If any service exceeds utilization 1.0, the queue at that service grows without bound and the system is unstable.

Mean Response Time (R_i)

The average time a request spends waiting in queue plus receiving service, derived from the M/M/1 formula:

$$R_i = S_i / (1 - \rho_i)$$

This is the Pollaczek–Khinchine (P-K) mean value formula for M/M/1. Note the non-linear relationship: as utilization approaches 1, response time diverges hyperbolically — this is the mathematical explanation of system collapse under overload.

Mean Queue Length (N_i)

The average number of requests present at the service center (both in service and waiting), derived via Little's Law:

$$N_i = \lambda_i \cdot R_i = \rho_i / (1 - \rho_i)$$

3.4 System-Level Performance Metrics

Two system-level aggregates characterize end-to-end user-perceived performance:

- System Throughput (X_s^{ys}): In steady state, the system throughput equals the external arrival rate: $X_s^{ys} = \lambda$. This holds as long as all service centers remain stable ($\rho_i < 1$ for all i).
- End-to-End System Response Time (R_s^{ys}): Weighted sum of individual response times, where the weight reflects the average number of visits per request:

$$R_s^{ys} = \sum_i V_i \cdot R_i$$

This formulation assumes requests traverse services sequentially (no parallel fan-out overlap benefit is modeled). This is a conservative assumption that slightly overestimates latency for services called in parallel (e.g., Frontend's concurrent calls).

4.0 Multi-Server Scaling: M/M/c Queueing

To evaluate horizontal scaling (pod replication), M/M/1 nodes are extended to M/M/c queues. When a service is scaled to c_i replicas, arriving requests are distributed across c_i parallel identical servers, each processing at the original rate $1/S_i$. This is the classic multi-server queue model.

4.1 M/M/c Derivations

For a service scaled to c_i replicas:

- Offered Load (traffic intensity): $a_i = \lambda_i \cdot S_i$
- Per-server utilization: $\rho_i = a_i / c_i$ (stability requires $\rho_i < 1$)

The Erlang-C formula gives the probability that an arriving request must wait in queue (all c_i servers are busy):

$$P_2 = C(c_i, a_i) = [a_i^{c_i} / (c_i! \cdot (1 - \rho_i))] / [\sum_{k=0}^{c_i-1} a_i^k / k! + a_i^{c_i} / (c_i! \cdot (1 - \rho_i))]$$

The mean response time for the M/M/c queue is then:

$$R_i = S_i + P_2 \cdot S_i / (c_i \cdot (1 - \rho_i))$$

The first term (S_i) is the service time; the second term is the expected waiting time in queue, which is proportional to P_2 and decreases as c_i increases or utilization decreases.

4.2 Horizontal Scaling Scenario: Frontend Saturation at 10 req/s

Consider the system under a 10 req/s workload. The Frontend service has an empirically measured demand of $S_0 = 116.0 \text{ ms} = 0.116 \text{ s}$ and a visit ratio $V_0 = 1.0$ (every external request visits it exactly once).

Unscaled State ($c_0 = 1$) — UNSTABLE

$$\rho_0 = 10 \times 1.0 \times 0.116 = 1.16 \geq 1.0$$

The server utilization exceeds 100%. The M/M/1 stability condition is violated.

Queue length grows without bound; response time $\rightarrow \infty$. The system collapses under this load.

Scaling the Frontend to 2 replicas ($c_0 = 2$):

- Offered load: $a_0 = 10 \times 1.0 \times 0.116 = 1.16$
- Per-server utilization: $\rho_0 = 1.16 / 2 = 0.58$ (58.0%) — well within the stability region.
- Erlang-C queueing probability (computed numerically): $P_z \approx 0.426$ (42.6% of requests must wait).
- Mean response time: $R_0 = 0.116 + 0.426 \times 0.116 / (2 \times 0.42) = 0.116 + 0.0588 \approx 0.1748 \text{ s} = 174.8 \text{ ms}$

Scaled State ($c_0 = 2$) — STABLE at 174.8 ms

A single additional replica mathematically transforms the system from unbounded collapse to a stable 174.8 ms response time — a finite, predictable, and acceptable operating point.

This result is verified by the REST API endpoint (Section 9).

5.0 Workload Characterization & Observability Stack

5.1 Observability Stack Architecture

Parameter extraction requires two complementary data streams: distributed traces for service-level timing and call-graph topology, and resource metrics for CPU utilization and container-level throughput.

The observability stack was built as follows:

- Locust Load Generator: Simulates realistic user sessions (browse, add to cart, checkout) against the Frontend HTTP interface.
- OpenTelemetry (OTel) Instrumentation: Each microservice was instrumented with OTel SDKs to emit distributed traces. Context propagation headers (traceparent) were manually configured in Go and Node.js gRPC interceptors to ensure spans are correctly linked across service boundaries.
- Jaeger Distributed Tracing (Port 16686): Receives, stores, and visualizes trace data. The Python parse_traces.py script walks the span tree to extract exclusive service demands (subtracting child span times) and routing probabilities (call frequency ratios).
- Prometheus (Port 9090): Scrapes time-series metrics from all containers. Used for CPU utilization and request-rate cross-validation.
- cAdvisor: Collects container-level resource metrics (CPU, memory, network I/O) and exposes them to Prometheus.

5.2 Parameter Extraction

Two categories of parameters are extracted from the telemetry:

- Exclusive Service Demand S_i : Extracted from Jaeger spans as the service's own processing time, net of child spans (i.e., time spent waiting for downstream calls is excluded). This is the correct input for the queueing model: we want the resource demand at service i , not the total latency seen by a caller.
- Routing Probabilities P_{ij} : Computed as the ratio of observed calls from service i to service j over total calls exiting service i . For example, if Frontend made 1000 total downstream calls and 350 went to ProductCatalog, then $P_{(Uob), (acdc)}^{Uob} = 0.35$.

5.3 Session Transition Matrix (P)

The routing matrix below shows the empirically measured transition probabilities. Rows represent the source service; columns represent the destination. Blank cells indicate no direct call relationship.

From \ To	Frontend	ProdCat	Currency	Cart	Recommend	Ad	Checkout	Shipping	Payment	Email
Frontend	—	0.35	0.20	0.15	0.10	0.08	0.05	0.05	—	—

From \ To	Frontend	ProdCat	Currency	Cart	Recommend	Ad	Checkout	Shipping	Payment	Email
CartService	—	—	—	—	—	—	—	—	—	— (→Redis 0.95)
Recommend.	—	0.80	—	—	—	—	—	—	—	—
Checkout	—	0.15	0.15	0.15	—	—	—	0.15	0.20	0.15

The CheckoutService row is notable: it distributes its outgoing calls across six downstream services with near-uniform probability (0.15 each), reflecting its role as the primary orchestrator in the checkout workflow. The CartService routes 95% of completions to Redis (not shown as a full column for brevity).

6.0 Software Architecture & Implementation

The modeling pipeline is implemented as a set of modular, decoupled Python components. Separation of concerns is deliberately enforced: mathematical solvers are isolated from data-collection scripts, and the REST API layer depends only on a compiled JSON parameter file, not on live telemetry connections.

6.1 Project Layout

SPE Project/

requirements.txt # Package dependencies (FastAPI, NumPy, Pandas, Matplotlib)

api/

app.py # FastAPI REST API — /solve endpoint with replica overrides

cli/

run_all.py # CLI Orchestrator: collection → model → validation

scripts/

model/

mva_solver.py # Core Open-Network MVA & Erlang-C solver classes

queueing_network.py # Parameter compiler, replica builder, sweep plots

queueing_network_parameters.json # Compiled parameters (auto-generated)

```
data_collection/  
  collect_metrics.py    # Prometheus scraper (resilient to offline servers)  
  parse_traces.py      # Jaeger trace parser — walks span trees  
validation/  
  compare.py           # RMSE / MAE / MAPE compiler & heatmap generator
```

6.2 Component Responsibilities

- `mva_solver.py`: Implements the MVA equations (Section 3) and Erlang-C formula (Section 4). Accepts a parameter dict and returns utilization, response time, queue length, and throughput for each service center.
- `queueing_network.py`: Reads empirical parameters, constructs the routing matrix P and demand vector S , invokes the MVA solver, and serializes results. On each run it writes `queueing_network_parameters.json` so the API can operate without a live telemetry connection.
- `parse_traces.py`: Queries the Jaeger HTTP API, recursively traverses span trees, computes exclusive durations, and tallies inter-service call counts to build the routing matrix.
- `collect_metrics.py`: Queries Prometheus for container-level CPU and request-rate metrics. Includes a dynamic SHA-hash-to-service-name resolver (see Section 8, Challenge 1).
- `app.py` (FastAPI): Exposes a POST `/solve` endpoint accepting `{"arrival_rate": float, "replicas": {service_name: int}}` and returns per-service and system-level predictions. Loads parameters from the JSON file at startup.
- `compare.py`: Loads analytical predictions and empirical measurements, computes RMSE/MAE/MAPE, and generates comparative bar charts and error heatmaps saved under `results/figures/`.

7.0 Empirical Validation & Results

The analytical model was solved at $\lambda = 5.0$ req/s and compared against empirical response times collected from the instrumented containerized deployment under the same load. All measurements represent steady-state averages collected after a warm-up period to eliminate transient effects.

7.1 Per-Service Response Time Validation

The table below shows predicted versus measured response times, along with absolute and relative errors, for all 11 service centers. Redis is excluded from error metrics as it is not directly observable through the HTTP/gRPC tracing path.

Service Centre	Predicted RT (ms)	Measured RT (ms)	Abs. Error (ms)	Rel. Error (%)
Frontend	276.19	285.00	−8.81	3.1%
ProductCatalogService	5.06	8.20	−3.14	38.4%
CurrencyService	3.01	5.10	−2.09	41.0%
CartService	4.01	6.80	−2.79	41.0%
RecommendationService	8.03	12.40	−4.37	35.2%
AdService	6.01	9.10	−3.09	33.9%
CheckoutService	15.06	22.50	−7.44	33.1%
ShippingService	4.00	7.20	−3.20	44.4%
PaymentService	10.01	14.30	−4.29	30.0%
EmailService	12.01	18.00	−5.99	33.3%
Redis	2.00	N/A	—	—

The Frontend prediction is highly accurate (3.1% relative error), which is the most critical result: the bottleneck service is correctly identified and quantified. The M/M/1 model systematically under-predicts response times for lightly-loaded backend services (30–44% relative error). This is expected and is analyzed in Section 10.3 — small absolute latencies (2–7 ms) are dominated by network stack overheads not captured in the model.

7.2 Compiled Error Metrics

Metric	Description	Value
RMSE	Root Mean Squared Error	4.98 ms
MAE	Mean Absolute Error	4.52 ms
MAPE	Mean Absolute Percentage Error	33.34%
E2E Predicted	System Response Time (Analytical)	282.42 ms
E2E Measured	System Response Time (Empirical)	292.15 ms
E2E Deviation	Absolute Deviation	−9.73 ms (3.3%)

The RMSE of 4.98 ms and MAE of 4.52 ms are both dominated by the absolute errors on CheckoutService (7.44 ms) and EmailService (5.99 ms) — services with moderate response times where the model is least accurate. The 33% MAPE is misleading as a top-line number: it is driven by the large relative errors on services with very small absolute response times (2–8 ms), where a 3 ms miss represents a 40% error. For the bottleneck and system-level metrics, accuracy is high.

7.3 Sensitivity & Saturation Sweep

The model was evaluated across arrival rates $\lambda \in [1, 10]$ req/s. The Frontend reaches saturation ($\rho = 1.0$) at the critical load:

$$\lambda_{\text{Tax}} = 1 / (V_0 \cdot S_0) = 1 / (1.0 \times 0.116) \approx 8.62 \text{ req/s}$$

Beyond 8.62 req/s, the Frontend queue length grows without bound and measured response times confirm empirically: requests begin to time out and the system becomes unresponsive. This saturation point was verified experimentally by the Locust load generator, validating the model’s predictive precision.

Practical Implication

The saturation threshold of 8.62 req/s corresponds to a safety margin of ~14% below 10 req/s. In production capacity planning, a typical target utilization is $\rho \leq 0.70$, implying a comfortable operating ceiling of approximately 6.0 req/s per Frontend replica. This directly informs auto-scaling trigger thresholds in Kubernetes HPA configurations.

8.0 Technical Challenges & Solutions

Challenge 1: Docker Container Namespace Resolution under WSL 2

Problem: cAdvisor queries the WSL 2 Linux kernel namespace. In Windows Docker Desktop environments, cAdvisor cannot resolve human-readable container names due to virtualization namespace isolation, exporting raw SHA-256 content hashes instead (e.g., `container_cpu_usage_seconds_total{id="sha256:3a8f..."}`). This made it impossible to map CPU metrics to named services.

Solution: `collect_metrics.py` was modified to query the Jaeger runtime metadata API, which maintains an active mapping between running process names and their container IDs. This produced a dynamic resolution dictionary mapping SHA hashes back to service names, restored without requiring elevated privileges or kernel namespace access.

Challenge 2: OTEL Context Propagation Disconnection

Problem: Early distributed traces showed disconnected root spans at language boundaries. Go services would emit spans with no parent, and Node.js backends would emit their own root spans, rather than child spans linked to the caller's trace context. This resulted in underestimated routing weights (missing call edges) and incorrect exclusive demand calculations.

Solution: The W3C `TraceContext` `traceparent` header was manually injected into gRPC client interceptors in both the Go and Node.js services. This ensured that the trace context was propagated through gRPC metadata across all service calls, producing correctly linked parent-child span trees in Jaeger and accurate routing probability extraction.

Challenge 3: Parameter Scoping for REST API Decoupling

Problem: The FastAPI `/solve` endpoint needs access to model parameters (S_i , P_{ij} , V_i) at runtime to evaluate new workload scenarios, but the original modeling scripts only saved output predictions, not input parameters. This created a tight coupling requiring live telemetry access at every API call.

Solution: A dedicated parameter compiler scope was added to `queueing_network.py` that serializes the full parameter set to `queueing_network_parameters.json` upon model execution. The API server loads this file at uvicorn startup, operating entirely independently of the telemetry stack. The JSON file acts as a contract between the offline modeling pipeline and the online prediction server.

9.0 Replication & Reproducibility Guide

The entire workflow is executable via two stages. All outputs are deterministic given the same empirical parameters.

Stage A: End-to-End Modeling CLI

This command installs dependencies, scrapes telemetry (or uses cached parameters), runs the MVA solver, generates utilization sweep curves, and compiles the validation comparison report:

```
# Install dependencies
pip install -r requirements.txt
# Run full analytical workflow at 5.0 req/s
python cli/run_all.py --arrival 5.0
```

Outputs written to `results/figures/comparison_summary.txt` and comparative charts saved under `results/figures/`.

Stage B: REST API Scaling Predictions

Start the FastAPI server and query it for horizontal scaling predictions:

```
# Start the prediction server
python -m uvicorn api.app:app --host 127.0.0.1 --port 8000

# Query with 2x Frontend replicas at 10 req/s
import requests
payload = {"arrival_rate": 10.0, "replicas": {"Frontend": 2}}
r = requests.post("http://127.0.0.1:8000/solve", json=payload)
print(r.json())
# Returns: Frontend response time = 174.8 ms (Erlang-C M/M/c)
```

10. Future Work & Recommendations

While the Open M/M/1 and M/M/c queueing network approximations are computationally efficient and highly accurate for bottleneck identification and saturation analysis, they operate under several simplifying assumptions. Three concrete extensions are recommended for subsequent academic work:

10.1 Layered Queueing Networks (LQN)

Limitation: M/M/1 assumes service centers are fully independent. In reality, synchronous microservice communication is blocking: the calling thread on the Frontend is held waiting for downstream responses (e.g., CheckoutService waiting for PaymentService). This results in nested resource holding — a thread at service i holds a CPU slot idle while waiting for service j , effectively reducing the true capacity of service i .

Recommendation: Construct a Layered Queueing Network (LQN) model using solvers such as LQNS (Layered Queueing Network Solver). LQN explicitly represents both software resources (thread pools, connection pools) and physical resources (CPU, disk) as layered servers. This accurately captures thread-pool exhaustion scenarios where a service crashes even when CPU utilization remains low — a common failure mode in production gRPC services.

10.2 Connection Pool and Thread Contention Modeling

Limitation: Containerized microservices enforce maximum thread counts and database connection pool limits (e.g., gRPC worker thread pools, Redis client pools). Under heavy load, requests queue not at the CPU but while waiting to acquire an available connection from the pool. The M/M/1 model does not represent this constraint.

Recommendation: Introduce a passive delay server or closed sub-network modeling node representing connection acquisition time. The demand S^c for connection acquisition can be derived from instrumentation (measuring time-to-first-byte after connection request) or modeled as a closed multi-class queue nested within the service center. This is particularly important for CartService (Redis connection pool) and CheckoutService (multiple downstream connection pools).

10.3 Network Transit and Serialization Demands

Limitation: The model showed a 33% MAPE error for lightly-loaded backend services. Investigation reveals this is largely attributable to absolute network transit latencies (2–7 ms) dominated by WSL 2

virtual network stack overhead and gRPC Protocol Buffers serialization/deserialization cycles. These costs are not captured in the exclusive service demands extracted from Jaeger spans.

Recommendation: Decompose the effective service demand into three additive components:

$$S'_i = S_{\text{processing}} + S_{\text{serialization}} + S_{\text{network_transit}}$$

Where $S_{\text{network_transit}}$ is estimated by measuring TCP/IP round-trip time (RTT) on the Docker overlay network, and $S_{\text{serialization}}$ is characterized by benchmarking Protobuf parsing CPU cycles for representative message payloads. Adding these residual demands would significantly reduce MAPE for backend services while leaving bottleneck prediction accuracy unchanged.

11. Conclusions

This project successfully formulated, implemented, and validated an analytical Open Queueing Network model for the 11-service Google Online Boutique microservice benchmark. The key contributions and outcomes are:

- Empirical Parameterization: Service demands and routing probabilities were successfully extracted from a live instrumented deployment using OTel, Jaeger, and Prometheus, including resolution of WSL 2 container namespace complications.
- Bottleneck Identification: The MVA solver precisely identified the Frontend as the system bottleneck with only 3.1% relative error on its response time prediction.
- Saturation Point Prediction: The critical load threshold was computed analytically as $\lambda^{\text{Tax}} \approx 8.62$ req/s, which was empirically confirmed by observing system timeout behavior under 10 req/s.
- Horizontal Scaling Validation: The M/M/c Erlang-C extension proved mathematically sound, demonstrating that a single additional Frontend replica stabilizes the system at 174.8 ms under a 10 req/s load that would otherwise collapse the system.
- REST API Deployment: A FastAPI server exposes the analytical model as a prediction endpoint, enabling programmatic capacity planning and integration into CI/CD pipelines or autoscaling controllers.

The 33% MAPE on lightly-loaded backend services represents a known and quantified limitation of the M/M/1 approximation for small absolute latencies dominated by network overhead. For the

primary engineering goal — identifying and sizing bottlenecks — the model is highly effective and computationally inexpensive.

This project demonstrates that classical analytical queueing models remain powerful, practical tools for modern cloud capacity planning, and that the integration of automated observability pipelines with analytical solvers represents a viable and efficient approach to continuous performance engineering.

References

1. Lazowska, E. D., Zahorjan, J., Graham, G. S., & Sevcik, K. C. (1984). Quantitative System Performance: Computer System Analysis Using Queueing Network Models. Prentice-Hall, Inc.
2. Bolch, G., Greiner, S., de Meer, H., & Trivedi, K. S. (2006). Queueing Networks and Markov Chains: Modeling and Performance Evaluation with Computer Science Applications. John Wiley & Sons.
3. Menasce, D. A., Almeida, V. A., & Dowdy, L. W. (2004). Performance by Design: Computer Capacity Planning by Example. Prentice Hall.
4. OpenTelemetry Project. (2024). Distributed Tracing Context Propagation. <https://opentelemetry.io/docs/concepts/signals/traces/>
5. Google Cloud Platform. (2024). Online Boutique Microservices Demo. <https://github.com/GoogleCloudPlatform/microservices-demo>